

Java Backend Architecture

Interview Series

Chapter 13.9

JMH Benchmarking

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

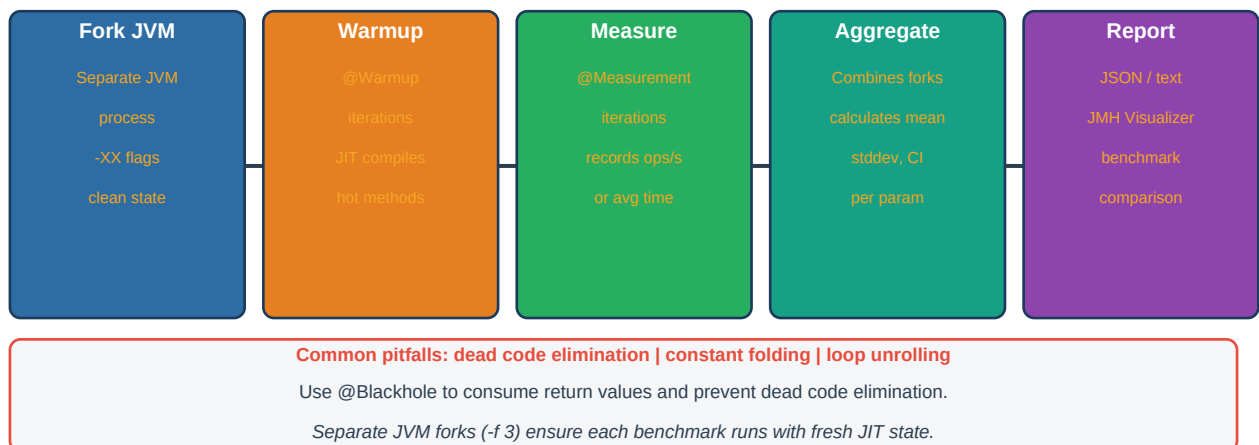
JMH Benchmarking

JMH (Java Microbenchmark Harness), developed by the JVM Performance Engineering team at Oracle, is the authoritative framework for writing and running Java microbenchmarks. Microbenchmarking is notoriously difficult because the JVM performs aggressive optimizations: JIT compilation eliminates dead code, folds constants, unrolls loops, and inlines methods in ways that can make a benchmark measure zero actual work. Writing a correct microbenchmark requires understanding these optimizations and using JMH annotations (`@Blackhole`, `@State`, `@Warmup`, `@Measurement`, `@Fork`) to prevent them from distorting results.

JMH runs benchmarks in a controlled lifecycle: separate JVM forks ensure clean JIT state per benchmark; warmup iterations allow JIT compilation to reach steady state (C2-compiled code); measurement iterations record the stable performance. Benchmark modes include Throughput (ops/s), AverageTime (ns/op), SampleTime (full latency histogram), and SingleShotTime (cold start cost). `@Param` annotations enable parameterized benchmarks across multiple input sizes. JMH produces JSON output that feeds into the JMH Visualizer for visual comparison. Results include mean, standard deviation, and confidence intervals — all essential for statistically valid conclusions.

JMH Benchmark Execution Lifecycle

JMH Benchmark Execution Lifecycle



Key Definitions

JMH	Java Microbenchmark Harness. Framework for writing statistically valid Java microbenchmarks, handling JIT warm-up, dead code elimination, and isolation.
@Benchmark	Marks a method as a benchmark entry point. JMH generates a harness class that calls this method in a tight measurement loop.
@BenchmarkMode	Specifies what to measure: Throughput (ops/s), AverageTime (time/op), SampleTime (latency histogram), or SingleShotTime (one invocation).
@Warmup	Configures warm-up iterations (e.g., 5 iterations of 1 second each) to allow JIT compilation to reach steady state before measurement.

@Measurement	Configures measurement iterations (e.g., 10 iterations of 1 second). JMH records ops/time per iteration and computes statistics.
@Fork	Runs the benchmark in N separate JVM processes. Eliminates JIT pollution between benchmarks. Recommended <code>@Fork(3)</code> for publication-quality results.
@State	Marks a class as providing benchmark state. <code>Scope.Thread</code> : one instance per thread. <code>Scope.Benchmark</code> : shared. <code>Scope.Group</code> : shared within a group.
@Blackhole	Consumes computed values to prevent dead code elimination. Pass results to <code>Blackhole.consume()</code> or declare <code>Blackhole</code> as benchmark parameter.
@Param	Parameterizes a benchmark over multiple values (e.g., array sizes [10, 100, 1000]). JMH runs the benchmark for each combination.
@Setup/@TearDown	Methods called before/after benchmark iterations. <code>Level.Trial</code> (once per fork), <code>Level.Iteration</code> (per iteration), <code>Level.Invocation</code> (per call).
Dead Code Elimination	JIT optimization that removes computations whose results are never used. Eliminates entire benchmark body if return value is unused.
Constant Folding	JIT replaces expressions involving constants with precomputed values at compile time, making benchmarks appear faster than real usage.
Loop Unrolling	JIT optimization reducing loop iteration overhead by duplicating loop body. Use <code>@OperationsPerInvocation</code> to account for manual loop counts.
JIT Warmup	JVM starts interpreting bytecode; after ~1000 invocations C1 compiles; after ~10000 C2 (server compiler) compiles with full optimization.
HDR Histogram	High Dynamic Range Histogram. Used by JMH <code>SampleTime</code> mode and <code>wrk2</code> to record latency distributions accurately across wide ranges without significant memory.

JMH Benchmark Modes

JMH Benchmark Modes

Throughput ops/s Higher = better Default mode Iterations/sec	AverageTime time/op Lower = better Avg per call ns/us/ms	SampleTime histogram time/op Captures percentiles	SingleShot one call No warmup Cold start cost
---	---	--	--

@State Scopes: `Scope.Thread` (one instance per thread, default) |

`Scope.Benchmark` (one instance shared across all threads) |

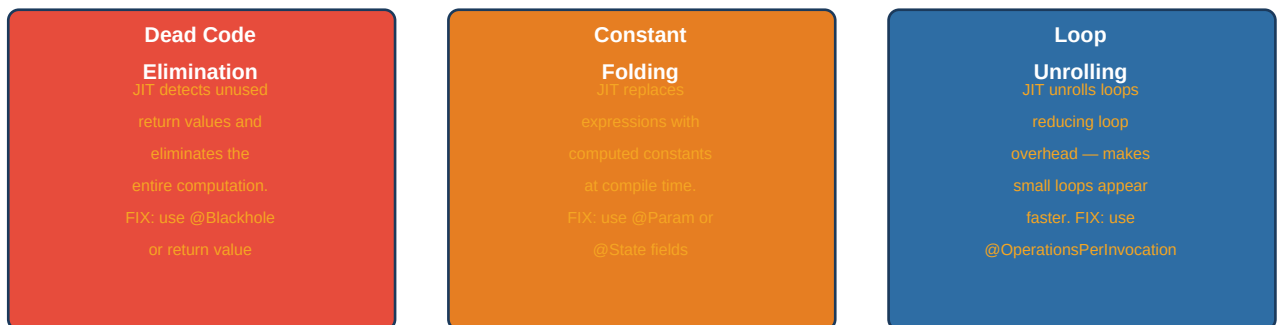
`Scope.Group` (shared within a benchmark group of threads)

Benchmark Mode Quick Reference

Mode	Annotation	Unit	Use Case	Higher/Lower Better
Throughput	@BenchmarkMode(Mode.Throughput)	ops/s	Maximizing operations per second	Higher is better
AverageTime	@BenchmarkMode(Mode.AverageTime)	ns/op	Minimizing per-call latency	Lower is better
SampleTime	@BenchmarkMode(Mode.SampleTime)	ns/op	Latency percentile distribution	Lower is better
SingleShot	@BenchmarkMode(Mode.SingleShotTime)	ops	Cold-start cost, no warmup	Lower is better
All	@BenchmarkMode(Mode.All)	all	Full picture, slower execution	Context-dependent

JIT Optimization Effects

JIT Optimization Effects on Microbenchmarks



Warmup iterations allow JIT to reach steady state (C2 compilation after ~10k invocations).

Without warmup, benchmarks measure JIT compilation overhead, not steady-state performance.

Annotated Code Examples

1. Basic JMH benchmark setup

```
import org.openjdk.jmh.annotations.*; import org.openjdk.jmh.infra.Blackhole; import
java.util.concurrent.TimeUnit; @BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS) @Warmup(iterations = 5, time = 1, timeUnit =
TimeUnit.SECONDS) @Measurement(iterations = 10, time = 1, timeUnit = TimeUnit.SECONDS) @Fork(3)
// 3 separate JVM processes for clean JIT state @State(Scope.Thread) public class
StringConcatBenchmark { private String prefix = "Hello"; private String suffix = "World";
@Benchmark public String stringPlus() { return prefix + " " + suffix; // Returns value — no
dead code elimination } @Benchmark public void stringBuilder(Blackhole bh) { String result =
new StringBuilder().append(prefix).append(" ").append(suffix).toString();
bh.consume(result); // Blackhole prevents DCE } }
```

2. @Param for parameterized benchmarks

```
@BenchmarkMode(Mode.Throughput) @OutputTimeUnit(TimeUnit.SECONDS) @Warmup(iterations = 3, time
= 1) @Measurement(iterations = 5, time = 1) @Fork(2) @State(Scope.Thread) public class
SortBenchmark { @Param({"100", "1000", "10000"}) public int size; // JMH runs benchmark for
each param value private int[] data; @Setup(Level.Invocation) // reset data before each call
public void setup() { data = new Random().ints(size).toArray(); } @Benchmark public void
arraySort(Blackhole bh) { Arrays.sort(data); bh.consume(data[0]); // consume to prevent DCE }
@Benchmark public void parallelSort(Blackhole bh) { Arrays.parallelSort(data);
bh.consume(data[0]); } }
```

3. SampleTime mode for latency percentiles

```
@BenchmarkMode(Mode.SampleTime) @OutputTimeUnit(TimeUnit.MICROSECONDS) @Warmup(iterations = 5,
time = 2) @Measurement(iterations = 10, time = 5) // longer for good histogram coverage
@Fork(2) @State(Scope.Thread) public class DatabaseQueryBenchmark { private DataSource
dataSource; @Setup(Level.Trial) public void setupDataSource() { HikariConfig config = new
HikariConfig(); config.setJdbcUrl("jdbc:postgresql://localhost/bench");
config.setMaximumPoolSize(10); dataSource = new HikariDataSource(config); } @Benchmark public
void selectByPrimaryKey(Blackhole bh) throws Exception { try (Connection conn =
dataSource.getConnection(); PreparedStatement ps = conn.prepareStatement( "SELECT * FROM
orders WHERE id = ?")) { ps.setLong(1, ThreadLocalRandom.current().nextLong(1, 100000)); try
(ResultSet rs = ps.executeQuery()) { if (rs.next()) bh.consume(rs.getString(1)); } } }
@TearDown(Level.Trial) public void tearDown() { ((HikariDataSource)dataSource).close(); } }
```

4. @State Scope.Benchmark — shared mutable state

```
// Scope.Benchmark: ONE state instance shared across all benchmark threads // Useful for
testing concurrent data structure performance @State(Scope.Benchmark) public class
ConcurrentMapState { public ConcurrentHashMap<Integer, String> map = new
ConcurrentHashMap<>(); @Setup(Level.Trial) public void populate() { IntStream.range(0,
100_000) .forEach(i -> map.put(i, "value_" + i)); } } @BenchmarkMode(Mode.Throughput)
@OutputTimeUnit(TimeUnit.SECONDS) @Warmup(iterations = 5) @Measurement(iterations = 10)
@Fork(3) @Threads(8) // 8 concurrent threads public class ConcurrentMapBenchmark { @Benchmark
public String concurrentRead(ConcurrentMapState state) { int key =
ThreadLocalRandom.current().nextInt(100_000); return state.map.get(key); } @Benchmark public
String computeIfAbsent(ConcurrentMapState state) { int key =
ThreadLocalRandom.current().nextInt(200_000); return state.map.computeIfAbsent(key, k ->
"new_" + k); } }
```

5. JMH Maven archetype and running benchmarks

```
# Create JMH project with Maven archetype mvn archetype:generate \ -DinteractiveMode=false \
-DarchetypeGroupId=org.openjdk.jmh \ -DarchetypeArtifactId=jmh-java-benchmark-archetype \
-DarchetypeVersion=1.37 \ -DgroupId=com.example \ -DartifactId=benchmarks \ -Dversion=1.0 #
Build fat JAR containing all benchmark classes mvn clean package -q # Run all benchmarks java
-jar target/benchmarks.jar # Run specific benchmark class/method java -jar
target/benchmarks.jar StringConcatBenchmark # Override annotations via CLI flags java -jar
target/benchmarks.jar \ -f 3 -wi 5 -i 10 -tu ns \ -bm avgt \ -rf json -rff results.json # Open
results.json at: https://jmh.morethan.io/ # for visual bar chart comparison
```

6. Preventing dead code elimination and constant folding

```
// WRONG: JIT sees result is never used => eliminates entire method body @Benchmark public void
badBenchmark() { Math.sqrt(12345.6789); // result discarded – JIT may eliminate this! } //
CORRECT approach 1: return the result @Benchmark public double goodReturn() { return
Math.sqrt(12345.6789); // JMH harness consumes return value } // CORRECT approach 2: use
Blackhole.consume() @Benchmark public void goodBlackhole(Blackhole bh) {
bh.consume(Math.sqrt(12345.6789)); } // WRONG: constant folding – JIT pre-computes 10 * 20 =
200 at compile time @Benchmark public int constantFolding() { return 10 * 20; } // JIT replaces
with: return 200; } // CORRECT: use @State fields so JIT cannot constant-fold
@State(Scope.Thread) public class MathState { public int a = 10; public int b = 20; }
@Benchmark public int noConstantFolding(MathState s) { return s.a * s.b; } // JIT cannot fold –
values read from state }
```

Interview Q&A;

Q1. Why is writing a correct Java microbenchmark difficult?

The JVM performs aggressive optimizations that can invalidate naive benchmarks: (1) Dead code elimination removes computations whose results are never used, making the benchmark measure zero work. (2) Constant folding precomputes constant expressions at JIT compile time. (3) Loop unrolling reduces loop overhead, making small loops appear faster. (4) JIT warmup means initial iterations are interpreted (slow) then C1/C2 compiled (fast) — without warmup you measure compilation overhead. (5) CPU caches and branch predictors are warm after repeated calls, not reflecting cold-path performance. JMH handles all of these correctly.

Q2. What does `@Fork` do and why is it important?

`@Fork(N)` runs each benchmark method in N separate JVM processes. This is critical because JIT compilation state from one benchmark contaminates subsequent ones in the same JVM. A benchmark measured after a similar benchmark may benefit from already-compiled common methods, showing artificially good results. Separate forks ensure each benchmark starts with a clean JIT state. For publication-quality results use `@Fork(3)` or more. `@Fork(0)` disables forking (useful for debugging in IDE, never for real measurement).

Q3. What is dead code elimination and how does `@Blackhole` prevent it?

Dead code elimination is a JIT optimization that removes code whose output is never observed. If a benchmark computes a value but never uses it, the JIT compiler may eliminate the entire computation. `@Blackhole` is a JMH-provided sink that the JIT cannot optimize away — it is implemented with volatile reads/writes to prevent elimination. Pass computed values to `bh.consume(result)` or declare `Blackhole` as a benchmark parameter. Alternatively, simply return the computed value (JMH harness consumes it).

Q4. What is the difference between `@Warmup` and `@Measurement` annotations?

`@Warmup` configures warm-up iterations: these are run before measurement starts, allowing JIT compilation (C1 at ~1000 invocations, C2 at ~10000) to reach steady state. Warmup results are discarded. `@Measurement` configures measurement iterations: these are timed and recorded. Typical settings: `@Warmup(iterations=5, time=1s) + @Measurement(iterations=10, time=1s)`. More iterations reduce variance; longer times sample more invocations per iteration.

Q5. What is `@State` and what are the scope options?

`@State` marks a class that provides benchmark state (setup data, objects to benchmark). `Scope.Thread`: each benchmark thread gets its own instance — no contention, default choice for most benchmarks. `Scope.Benchmark`: all threads share one instance — used for testing concurrent data structures or shared caches. `Scope.Group`: shared within a named benchmark group (used with `@Group` for producer-consumer benchmarks). `@Setup` and `@TearDown` methods initialize and clean up state.

Q6. What benchmark mode should you use for latency vs throughput measurement?

For throughput (how many operations per second): `Mode.Throughput`. For latency (how long does one operation take on average): `Mode.AverageTime`. For latency distribution (p50/p95/p99): `Mode.SampleTime` — JMH records every individual invocation time and produces a full histogram. For cold-start cost (time for a single invocation with no warmup): `Mode.SingleShotTime`. Use `Mode.All` to run all modes in one benchmark (slower but comprehensive).

Q7. What is constant folding and how do you avoid it in benchmarks?

Constant folding is a JIT optimization that precomputes expressions involving constants at compile time. A benchmark computing `10 * 20` becomes `return 200` after JIT optimization — measuring zero actual work. Avoid it by storing inputs in `@State` fields (JIT cannot predict field values at compile time). Annotate fields without `final` to ensure JIT does not treat them as compile-time constants. Use `@Param` to vary inputs across multiple values.

Q8. What is @Param and when should you use it?

@Param allows declaring multiple values for a benchmark field. JMH runs the full benchmark (all forks, warmups, measurements) for every parameter combination. Example: `@Param({"10","100","1000"}) int size;` benchmarks the algorithm at three input sizes. This is essential for algorithmic complexity analysis and for finding performance cliffs (e.g., cache misses above a certain size). Parameter combinations multiply execution time — keep the number of combinations manageable.

Q9. What is @Setup and @TearDown and at what levels can they run?

@Setup annotates a method called before benchmark execution to initialise state. @TearDown annotates cleanup. Level.Trial: called once per fork (once per JVM process) — use for expensive setup (open database connections, load test files). Level.Iteration: called before each measurement/warmup iteration — use for resetting state between timing windows. Level.Invocation: called before every single benchmark method call — use only for per-call setup; adds overhead and should be used carefully.

Q10. How do you run JMH benchmarks and where do you view results?

Build a fat JAR with Maven (mvn clean package using the JMH archetype pom.xml) then run: `java -jar benchmarks.jar`. Results print to stdout as a text table. For JSON output: `-rf json -rff results.json`. Open results.json at jmh.morethan.io (JMH Visualizer) for bar charts comparing benchmark scores. CLI flags override annotations: `-f` (forks), `-wi` (warmup iterations), `-i` (measurement iterations), `-bm` (mode), `-tu` (time unit).

Q11. What does @OperationsPerInvocation do?

@OperationsPerInvocation(N) tells JMH that each invocation of the benchmark method performs N logical operations. Without it, if your benchmark loops 1000 times internally, JMH reports throughput for the entire loop as one operation. With @OperationsPerInvocation(1000), JMH divides the result by 1000 to report per-element throughput. This is especially important when benchmarking collection processing loops to get meaningful per-element metrics.

Q12. Why should you never benchmark inside an IDE without forking?

Running JMH benchmarks inside an IDE with @Fork(0) shares the JVM with IDE and other running code, polluting JIT compilation state and CPU caches. The IDE itself (code completion, background indexing) competes for CPU and memory. Results may vary by 2-10x compared to a clean forked JVM. Always use @Fork(1) or more for real measurements. Use @Fork(0) only for debugging benchmark setup (checking for compile errors, verifying @State setup).

Q13. What is JMH Visualizer and what does it show?

JMH Visualizer (jmh.morethan.io) is a web tool that accepts JMH JSON output and renders bar charts comparing benchmark scores with error bars. It shows: benchmark name on the x-axis, score (ops/s or time/op) on the y-axis, error bars representing the 99.9% confidence interval. Multiple benchmarks on the same chart enable visual comparison. Use it to present before/after optimization comparisons or algorithm choice trade-offs.

Q14. How do you handle benchmark variance and statistical validity?

Use multiple forks (@Fork(3)+) to get independent JVM samples. Use sufficient measurement iterations and time to get stable results (low coefficient of variation). JMH reports score error (confidence interval at 99.9%) — if the error is larger than the difference you are trying to measure, results are not statistically significant. Increase iterations/time to reduce variance. Variance also comes from GC pauses: pin GC with `-XX:+UseParallelGC -Xmx4g -Xms4g` for minimum GC interference.

Q15. What JMH pitfall arises with String operations?

String concatenation benchmarks are vulnerable to JIT string interning and constant folding. If the input strings are compile-time constants, JIT may pre-compute the concatenation. Always use `@State` fields for string inputs. Also, `String.intern()` has complex JIT behaviour — results may differ dramatically between warmup and measurement phases. Use `@BenchmarkMode(Mode.AverageTime)` with `@Warmup(iterations=10)` to ensure fully JIT-compiled string operations are measured.

Q16. How does JMH handle multi-threaded benchmarks?

Use `@Threads(N)` to specify concurrent threads. Use `@State(Scope.Benchmark)` for shared state accessed by all threads. For producer-consumer scenarios use `@Group("name")` to define named groups and `@GroupThreads(N)` to allocate threads to each role. Use `@BenchmarkMode(Mode.Throughput)` for concurrent benchmarks to measure aggregate throughput. Use `Scope.Thread` for thread-local state (no sharing) to test single-thread performance without contention.

Q17. What JVM flags should you set when running JMH benchmarks?

For reproducible results: `-server` (64-bit server VM, usually default). To minimise GC interference: `-XX:+UseParallelGC -Xms4g -Xmx4g` (fixed heap size, parallel GC with minimal overhead). To disable adaptive compilation tiering for stable results: `-XX:-TieredCompilation` (uses C2 only). To control JIT inlining: `-XX:InlineSmallCode=N`. Pass extra JVM flags via `@Fork` annotation: `@Fork(jvmArgsPrepend = {"-XX:+UseParallelGC"})`.

Q18. What is the recommended JMH Maven dependency setup?

Add to `pom.xml`: `org.openjdk.jmh:jmh-core` and `org.openjdk.jmh:jmh-generator-annprocess` (annotation processor). The annotation processor generates the benchmark harness classes at compile time. Use the `maven-shade-plugin` or the JMH archetype `pom` to produce a fat executable JAR. The archetype `pom.xml` is the easiest starting point: `mvn archetype:generate` with `archetypeArtifactId=jmh-java-benchmark-archetype`. Current stable version is 1.37.

Q19. How do you benchmark Java serialization frameworks?

Create `@State` with a pre-built object graph (Level.Trial setup). Benchmark `ser/deser` separately. Use `@Param` to vary payload sizes and nesting depths. For throughput comparison of Jackson vs Gson vs Protobuf: return the `byte[]` or `String` result to prevent DCE. For deserialization: read bytes from `@State` (pre-serialized), avoid I/O in the benchmark path. `@Fork(3) @Warmup(5) @Measurement(10)` is a good starting config. Record both ops/s (`Mode.Throughput`) and bytes/s (multiply by payload size).

Q20. What is the difference between microbenchmark results and production performance?

Microbenchmarks measure isolated code paths in a controlled environment with warmed JIT, pre-filled caches, and no I/O. Production performance includes: JVM startup cost, cold JIT state, contention with other services on the same host, GC pauses, network I/O, thread scheduling jitter, cache misses from diverse access patterns, and system call overhead. Microbenchmarks are valuable for comparing algorithmic choices and data structure implementations but should always be validated against load tests with realistic traffic patterns before drawing production conclusions.